



Name: 1. MUHAMMAD FAUZAAN BIN NAZRUDDIN 2. DANIAL AMIR ASYRAF BIN SAIFUDDIN 3. MUHAMAD FAHMI AKMAL BIN MOHD ZIN		
Student No: 1. AM2512021170 2. AM2512021021 3. AM2512021804		
Lecturer: MADAM NURUL AZLIN LIYANA BINTI MOHAMAD REDZUAN	Semester: 03	Section: 01
Course Code and Course Name: SWc2623 – Software Development Paradigms	Submission Date: 30 th SEPTEMBER 2026	
Assignment Title: ASSIGNMENT	Extension & Late submission: Allowed / <u>Disallowed</u>	
Assignment Type: (circle) INDIVIDUAL / GROUP / FINAL ASSESSMENT	Assignment Mark	Returning Date:
<u>Penalties:</u> 1. <u>10% of the original mark will be deducted</u> for every one week period after the submission date 2. <u>No work</u> will be accepted <u>after two weeks</u> of the deadline 3. If you were unable to submit the coursework on time due to extenuating circumstances you may be eligible for an extension 4. Extension will <u>not exceed one week</u>		
Declaration: I/we the undersigned confirm that I/we have read and agree to abide by these regulations on plagiarism and cheating. I/we confirm that this piece of work is my/our own. I/we consent to appropriate storage of our work for checking to ensure that there is no plagiarism/ academic cheating.		
_____ MUHAMMAD FAUZAAN BIN NAZRUDDIN	_____ DANIAL AMIR ASYRAF BIN SAIFUDDIN	_____ MUHAMAD FAHMI AKMAL BIN MOHD ZIN
This section may be used for feedback or other information		

TABLE OF CONTENTS

1. Introduction.....	3
2. PART A: Functional Programming (Haskell) Learning Performance Analysis.....	4
3. Part B: Logic Programming (Prolog) - Module Advisory and Certification.....	6
4. Part C: Multi-Paradigm Programming (Python) - Integrated Learner Dashboard.....	9
5. Reference.....	14

1. INTRODUCTION

This report presents the design and implementation of the **Smart Learning Pathway and Certification System**, a prototype developed for a training centre to analyse learner performance, advise on module eligibility, and present an integrated learning profile. The system is implemented using three distinct programming paradigms: **Functional Programming (Haskell)**, **Logic Programming (Prolog)**, and **Multi-Paradigm Programming (Python)**.

The primary objective is to demonstrate how the same problem domain can be solved effectively using different computational approaches, each with its own strengths and design philosophies. **Part A** focuses on functional programming concepts such as immutability, pure functions, and higher-order functions to perform learning performance analysis. **Part B** employs logic programming to build a knowledge base of facts and rules that manages module advisory and certification queries through unification and inference. **Part C** combines object-oriented and functional techniques in Python to create an integrated learner dashboard with data processing and summary reporting capabilities.

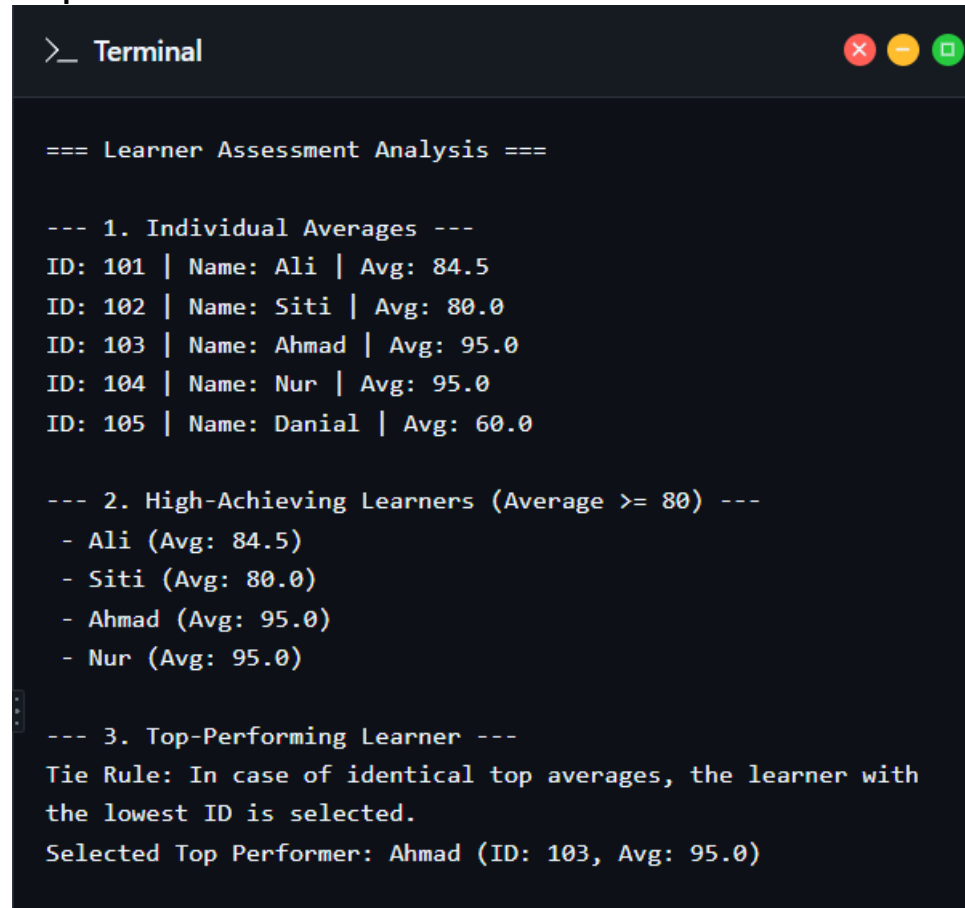
To ensure a valid comparison across the three paradigms, all parts utilise a consistent dataset of five learners (Ali, Siti, Ahmad, Nur, and Danial), six modules, and uniform prerequisite and certification rules. This consistency allows the report to evaluate how each paradigm addresses the same computational requirements while highlighting the unique characteristics of functional, logic, and multi-paradigm programming approaches.

2. PART A: Functional Programming (Haskell) Learning Performance Analysis

Introduction

This part of the project we develop a program with Haskell to analyse learner's assessment scores using functional programming.

Output



```
>_ Terminal

=== Learner Assessment Analysis ===

--- 1. Individual Averages ---
ID: 101 | Name: Ali | Avg: 84.5
ID: 102 | Name: Siti | Avg: 80.0
ID: 103 | Name: Ahmad | Avg: 95.0
ID: 104 | Name: Nur | Avg: 95.0
ID: 105 | Name: Danial | Avg: 60.0

--- 2. High-Achieving Learners (Average >= 80) ---
- Ali (Avg: 84.5)
- Siti (Avg: 80.0)
- Ahmad (Avg: 95.0)
- Nur (Avg: 95.0)

--- 3. Top-Performing Learner ---
Tie Rule: In case of identical top averages, the learner with
the lowest ID is selected.
Selected Top Performer: Ahmad (ID: 103, Avg: 95.0)
```

Functional Programming

Functional programming is a declarative programming paradigm that treats computation as the evaluation of mathematical functions. It avoids changing-state and mutable data heavily relying on immutability, pure functions, higher-order functions and types.

Core concepts of Functional Programming

Here are the examples for core concepts of functional programming demonstrated in the Haskell program.

1. Immutability and Pure Functions

In Haskell data is immutable by default once a data structure is created it cannot be modified in place. A pure function always yields identical outputs for identical inputs without altering external environments.

Example: `averageScore` safely computes and average from a `Learner` record without modifying the original dataset.

```
23 averageScore :: Learner -> Double
24 averageScore (Learner _ _ []) = 0.0
25 averageScore (Learner _ _ scores) = sum scores / fromIntegral (length scores)
26
```

2. Higher-order Functions

Functional programming replaces imperative loops with higher-order functions as arguments focusing on what to compute rather than how to iterate.

Example: `filter` uses an anonymous lambda predicate to extract high achievers precisely.

```
28 highAchievers :: [Learner] -> [Learner]
29 highAchievers = filter (\l -> averageScore l >= 80.0)
```

3. Pattern matching

Complex structures are broken down safely using pattern matching.

Example: `topLearner` pattern-matches an empty list edge-case and handles tie-breaking comparisons declaratively using `case`.

3. Part B: Logic Programming (Prolog) - Module Advisory and Certification

1. Introduction

This project develops a Prolog knowledge base to manage learners, modules, prerequisites, and certification rules. The knowledge base allows the system to check eligibility, recommend modules, and verify completion of requirements based on defined facts and rules.

2. Facts

The following facts are used in the knowledge base:

% Learners

```
learner(ali, computer_science).  
learner(siti, information_technology).  
learner(ahmad, computer_science).  
learner(nur, information_technology).  
learner(danial, software_engineering).
```

% Modules

```
module(programming).  
module(database).  
module(web_development).  
module(object_oriented_programming).  
module(data_structures).  
module(software_project).
```

% Completed modules

```
completed(ali, programming).  
completed(ali, database).  
completed(ali, web_development).  
completed(ali, object_oriented_programming).  
completed(ali, data_structures).  
completed(ali, software_project).  
completed(siti, programming).  
completed(siti, database).  
completed(siti, web_development).  
completed(ahmad, programming).  
completed(ahmad, object_oriented_programming).  
completed(nur, programming).  
completed(nur, database).  
completed(nur, object_oriented_programming).  
completed(nur, data_structures).  
completed(danial, programming).  
completed(danial, database).
```

% Prerequisites

```
prerequisite(database, programming).  
prerequisite(web_development, programming).  
prerequisite(object_oriented_programming, programming).
```

```
prerequisite(data_structures, programming).
prerequisite(data_structures, object_oriented_programming).
prerequisite(software_project, programming).
prerequisite(software_project, database).
prerequisite(software_project, web_development).
prerequisite(software_project, object_oriented_programming).
```

```
% Required modules
required_module(programming).
required_module(database).
required_module(web_development).
required_module(object_oriented_programming).
required_module(data_structures).
required_module(software_project).
```

3. Rules

```
eligible(Learner, Module) :-
  learner(Learner, _),
  module(Module),
  \+ completed(Learner,
  Module), forall(
  prerequisite(Module, Req),
  completed(Learner, Req)
  ).
```

```
recommended_module(Learner,
Module) :-eligible(Learner,
Module).
```

```
certification_eligible(Learner,
Module) :-completed(Learner,
Module).
```

```
eligible_for_certification(Learner
) :-learner(Learner, _),
forall(
required_module(M),
completed(Learner, M)
).
```

4. Sample Queries and Results

a) Successful and Unsuccessful Eligibility
?- eligible(ahmad, database).
true.

?- eligible(siti, data_structures).
false.

?- eligible(danial,
web_development).
true.

b) Recommendation
?- recommended_module(dania
l, M). M =

```
web_development ;
M =
object_oriented_programmi
ng ;
false.
c) Certification Check
?- certification_eligible(ali, software_project).
true.
```

```
?- certification_eligible(siti, software_project).
false.
```

```
d) Full Certification Eligibility
?- eligible_for_certification(ali).
true.
```

```
?- eligible_for_certification(siti).
false.
```

5. Explanation

Prolog is used in this project to build a knowledge base for module advisory and certification. The system stores information as facts, such as learner details, available modules, completed modules, prerequisites, and required modules. Rules are then used to derive answers from these facts. The eligible rule checks that a learner exists, the selected module exists, the module has not already been completed, and all its prerequisites have been completed. The recommended module rule uses the eligibility result to suggest modules to a learner. The certification eligible rule checks whether a particular module has already been completed, while eligible_for_certification checks all required modules at once. Different queries can be entered into SWI-Prolog to test successful and unsuccessful cases. This demonstrates the use of facts, rules, logical inference, and backtracking in solving an academic advisory problem.

6. Correction Made

One issue in the earlier version was the use of module (Module, _) in the eligibility rule. The module facts are defined with one argument, such as module(programming), so the rule has been corrected to use module (Module). This avoids an arity mismatch when the query is executed.

7. Conclusion

The Prolog knowledge base provides a simple way to manage module eligibility, recommendations, and certification requirements. By separating facts from rules, the system can answer several different queries using the same knowledge base. The eight sample queries cover successful and unsuccessful eligibility, module recommendations, individual module completion, and overall certification eligibility.

8. Reference

- SWI-Prolog. (n.d.). SWI-Prolog Reference Manual: Executing a Query. SWI-Prolog. SWI-Prolog Reference Manual
- SWI-Prolog. (n.d.). SWI-Prolog — forall/2. SWI-Prolog Documentation. SWI-Prolog forall/2 Documentation
- Blackburn, P., Bos, J., & Striegnitz, K. (n.d.). Learn Prolog Now! — Facts, Rules, and Queries. Learn Prolog Now!

4. **Part C: Multi-Paradigm Programming (Python) - Integrated Learner Dashboard**

1. Problem Description

This part develops an integrated learner dashboard using Python. The program combines object-oriented programming and functional programming techniques to analyse learner performance. It calculates average scores, classifies performance, filters high-achieving learners, ranks learners, and produces a summary table using the pandas library. The same five learners (Ali, Siti, Ahmad, Nur, Danial) and modules from Part B are used so that the three solutions remain consistent.

2. Design / Approach

The solution uses two programming paradigms together:

- **Object-Oriented Programming:** A Learner class is created to store learner ID, name, completed modules, and scores. The class includes input validation and methods to calculate the average score and classify performance.
- **Functional Programming:** Functions such as filter, map, reduce, and sorted (with lambda) are used to process collections of learners without changing the original data.
- **Library Integration:** The pandas library is used to generate a clean summary table of all learners.

This design keeps the data organised inside objects while using functional tools for data processing and analysis.

3. Selected Code Extracts

Learner class (Object-Oriented part):

```

25  ✓ def __init__(self, learner_id: str, name: str,
26          completed_modules: Optional[List[str]] = None,
27          scores: Optional[Dict[str, float]] = None):
28
29      if not isinstance(learner_id, str) or learner_id.strip() == "":
30          raise ValueError("learner_id must be a non-empty string")
31      if not isinstance(name, str) or name.strip() == "":
32          raise ValueError("name must be a non-empty string")
33
34      self.learner_id = learner_id.strip()
35      self.name = name.strip()
36
37      if completed_modules is None:
38          self.completed_modules = []
39      else:
40          self.completed_modules = list(completed_modules)
41
42      if scores is None:
43          self.scores = {}
44      else:
45          self.scores = dict(scores)
46
47      # Check every score is valid (0 to 100)
48      for module_name in self.scores:
49          score = self.scores[module_name]
50          if not isinstance(score, (int, float)):
51              raise ValueError("Score must be a number")
52          if score < 0 or score > 100:
53              raise ValueError("Score must be between 0 and 100")
54

```

```

55  ✓    def average_score(self) -> float:
56        if len(self.scores) == 0:
57            return 0.0
58
59        total = 0.0
60        count = 0
61        for score in self.scores.values():
62            total = total + score
63            count = count + 1
64
65        average = total / count
66        return average
67
68  ✓    def classify_performance(self) -> str:
69        avg = self.average_score()
70
71        if avg >= 85:
72            return "Excellent"
73        elif avg >= 80:
74            return "High Achiever"
75        elif avg >= 70:
76            return "Good"
77        elif avg >= 50:
78            return "Satisfactory"
79        else:
80            return "Needs Improvement"

```

Functional-style operations:

```

135    def get_high_achievers(learners: List[Learner]) -> List[Learner]:
136        """filter + lambda"""
137        result = list(filter(lambda l: l.average_score() >= 80, learners))
138        return result
139
140
141    def get_averages(learners: List[Learner]) -> List[float]:
142        """map + lambda"""
143        result = list(map(lambda l: l.average_score(), learners))
144        return result
153    def total_modules_completed(learners: List[Learner]) -> int:
154        """reduce + lambda"""
155        result = reduce(lambda acc, l: acc + len(l.completed_modules), learners, 0)
156        return result
157

```

4. Test Table

Test Case	Input	Expected Result	Actual Result	Pass/Fail	Test Case
Normal – Ali	Ali's full scores	avg ≈ 88.83, Excellent	avg = 88.83, Excellent	Pass	Normal – Ali
Boundary – average exactly 80	scores = {80, 80}	avg = 80.0, High Achiever	avg = 80.0, High Achiever	Pass	Boundary – average exactly 80
Invalid input – score > 100	score = 105	ValueError raised	ValueError raised	Pass	Invalid input – score > 100
Edge – no scores	empty scores	avg = 0.0, Needs Improvement	avg = 0.0, Needs Improvement	Pass	Edge – no scores
Functional – high achievers	full dataset	contains Ali and Nur	['Ali', 'Nur']	Pass	Functional – high achievers

5. Screenshots

```
=====
INTEGRATED LEARNER DASHBOARD
=====

--- Individual Learner Details ---

Ali (L001)
Completed modules : ['programming', 'database', 'web_development', 'object_oriented_programming', 'data_structures', 'software_project']
Scores            : {'programming': 88, 'database': 85, 'web_development': 90, 'object_oriented_programming': 92, 'data_structures': 87, 'software_project': 91}
Average          : 88.83
Classification    : Excellent

Siti (L002)
Completed modules : ['programming', 'database', 'web_development']
Scores            : {'programming': 78, 'database': 82, 'web_development': 75}
Average          : 78.33
Classification    : Good

Ahmad (L003)
Completed modules : ['programming', 'object_oriented_programming']
Scores            : {'programming': 70, 'object_oriented_programming': 68}
Average          : 69.0
Classification    : Satisfactory

Nur (L004)
Completed modules : ['programming', 'database', 'object_oriented_programming', 'data_structures']
Scores            : {'programming': 95, 'database': 88, 'object_oriented_programming': 91, 'data_structures': 89}
Average          : 90.75
Classification    : Excellent

Danial (L005)
Completed modules : ['programming', 'database']
Scores            : {'programming': 65, 'database': 72}
Average          : 68.5
Classification    : Satisfactory

--- High Achievers (average >= 80) ---
• Ali: 88.83 (Excellent)
• Nur: 90.75 (Excellent)

--- Ranked by Average Score ---
1. Nur      90.75 [Excellent]
2. Ali      88.83 [Excellent]
3. Siti     78.33 [Good]
4. Ahmad    69.0  [Satisfactory]
5. Danial   68.5  [Satisfactory]
```

Test Cases

```
=====
TEST CASES
=====

[Test 1] Normal - Ali average and classification
Expected : average = 88.83, classification = Excellent
Actual   : average = 88.83 , classification = Excellent
Result   : PASS

[Test 2] Boundary - average == 80
Expected : average = 80.0, classification = High Achiever
Actual   : average = 80.0 , classification = High Achiever
Result   : PASS

[Test 3] Invalid input - score > 100
Expected : ValueError raised
Actual   : Score must be between 0 and 100
Result   : PASS

[Test 4] Edge - no scores
Expected : average = 0.0, classification = Needs Improvement
Actual   : average = 0.0 , classification = Needs Improvement
Result   : PASS

[Test 5] Functional - high achievers
Expected : contains Ali and Nur
Actual   : ['Ali', 'Nur']
Result   : PASS
```

6. Explanation

This solution demonstrates how Python supports multiple programming paradigms in a single program. The object-oriented paradigm is used through the Learner class. The class encapsulates data (learner ID, name, completed modules, and scores) and behaviour (average calculation and performance classification). Input validation is also performed inside the constructor to ensure data integrity.

The functional paradigm is applied when processing collections of learners. The filter function combined with a lambda expression is used to extract high-achieving learners. The map function is used to obtain a list of average scores. The reduce function aggregates the total number of completed modules across all learners. The sorted function ranks learners by performance. These operations do not modify the original list of learners, which is consistent with functional programming principles.

By combining both paradigms, the program benefits from the structure and reusability of object-oriented design while using the clarity and expressiveness of functional tools for data processing. The pandas library is integrated only for presentation, producing a readable summary table. This multi-paradigm approach shows that Python can effectively combine different styles to solve a real problem in a clean and organised way.

7. Conclusion

The Python solution successfully implements an integrated learner dashboard. It meets all the requirements of Part C by combining a well-designed class structure with functional operations and library support. The program produces correct averages, performance classifications, rankings, and a summary table. The use of both object-oriented and functional paradigms demonstrates Python's flexibility as a multi-paradigm language.

5. REFERENCES

Haskell tutorial. Tutorials point. (n.d.). <https://www.tutorialspoint.com/haskell/index.htm>

TutorialsPoint. (n.d.). *Online Haskell Compiler & Ide - Free Haskell Programming Tool*. <https://www.tutorialspoint.com/compilers/online-haskell-compiler.htm>

Functional programming. Functional programming - HaskellWiki. (n.d.). https://wiki.haskell.org/index.php?title=Functional_programming

Haskell programming language -introduction | by Mearaf tadewos | medium. (n.d.-b). <https://medium.com/@mearaftadewos/haskell-programming-language-introduction-dac99443a10d>

Introduction. Introduction - Learn You a Haskell for Great Good! (n.d.). <https://learnyouahaskell.github.io/introduction.html>

SWI-Prolog. (n.d.). SWI-Prolog Reference Manual: Executing a Query. SWI-Prolog. SWI-Prolog Reference Manual

SWI-Prolog. (n.d.). SWI-Prolog — forall/2. SWI-Prolog Documentation. SWI-Prolog forall/2 Documentation

Blackburn, P., Bos, J., & Striegnitz, K. (n.d.). Learn Prolog Now! — Facts, Rules, and Queries. Learn Prolog Now!

Python Software Foundation. (2025). *About the Python Software Foundation*. <https://www.python.org/psf/>

Python Software Foundation. (2025). *Python 3 documentation*. <https://docs.python.org/3/>

Kuchling, A. M. (2025). *Functional programming HOWTO* (Python 3.10 documentation). Python Software Foundation. <https://docs.python.org/3.10/howto/functional.html>

McKinney, W. (2010). Data structures for statistical computing in Python. In S. van der Walt & J. Millman (Eds.), *Proceedings of the 9th Python in Science Conference* (pp. 56 -61). <https://doi.org/10.25080/Majora-92bf1922-00a>

The pandas development team. (2025). *pandas* [Computer software]. Zenodo. <https://doi.org/10.5281/zenodo.3509134>

The pandas development team. (2025). *pandas documentation*. NumFOCUS, Inc. <https://pandas.pydata.org/docs/>

ASSIGNMENT
SWC2623

STUDENT NAME _____
STUDENT NAME _____
STUDENT NAME _____

PART A: Functional Programming (Haskell)					
CRITERIA	Excellent (8–10 marks)	Good (5–7 marks)	Satisfactory (3–4 marks)	Poor (0–2 marks)	MARKS
Program correctness	All required functions implemented correctly; program runs flawlessly; accurate results for average ^{average} distinctions.	Minor logic or syntax issues; outputs mostly correct.	Major issues in logic or missing functionality; partial results.	Program fails to run or missing critical functions.	
Use of functional programming principles	Clear use of recursion, higher-order functions, and immutability. Demonstrates understanding of pure	Uses some functional features appropriately.	Limited use of functional concepts; mostly imperative.	No evidence of functional programming principles.	
Explanation (150 words)	Clearly explains how functional concepts were applied with relevant examples.	Explanation mostly clear but lacking detail or examples.	Minimal or unclear explanation.	Missing or irrelevant explanation.	
Part B: Logic Programming (Prolog)					
CRITERIA	Excellent (8–10 marks)	Good (5–7 marks)	Satisfactory (3–4 marks)	Poor (0–2 marks)	MARKS
Program correctness	All facts and rules implemented correctly; outputs for eligibility, recommendations, and graduation are accurate.	Mostly correct logic; minor issues with rule definitions or outputs.	Partial or inconsistent results; missing facts or rules.	Program does not run or rules are incorrect.	
Use of logic programming principles	Demonstrates excellent understanding of declarative problem-solving using facts and rules.	Appropriate use of logic principles with some minor flaws.	Limited or mechanical use of Prolog features.	No understanding of logic programming shown.	
Explanation (150 words)	Clear, detailed explanation linking logic programming concepts to implementation.	Mostly clear but lacks technical depth.	Minimal or vague explanation.	Missing or irrelevant explanation.	
Part C: Multi-Paradigm Programming (Python)					
CRITERIA	Excellent (8–10 marks)	Good (5–7 marks)	Satisfactory (3–4 marks)	Poor (0–2 marks)	MARKS
Program functionality	Fully implements class structure, functional operations, performance prediction, and library integration; outputs correct and relevant.	Minor bugs or incomplete features but demonstrates overall understanding.	Partial implementation; some errors or missing functionalities.	Does not meet requirements; fails to execute.	
Use of multiple paradigms	Demonstrates clear integration of OOP and functional paradigms (e.g., lambda, map/filter/reduce with class methods).	Reasonable integration with some imbalance between paradigms.	Limited use of multiple paradigms.	Only one paradigm used or no clear paradigm integration.	
Explanation (150–200 words)	Detailed and accurate explanation of how Python supports multiple paradigms, with examples.	Good explanation but lacks depth or examples.	Minimal or superficial explanation.	Missing or irrelevant explanation.	
Overall Report Quality					
CRITERIA	Excellent (8–10 marks)	Good (5–7 marks)	Satisfactory (3–4 marks)	Poor (0–2 marks)	MARKS
Report structure & submission	Report is well-organized, complete, and neatly formatted; includes group details, source code links, and screenshots.	Minor formatting or missing elements.	Incomplete report or missing one major section.	Disorganized or missing major deliverables.	